LITE MTP: MINIMALIST MULTI-TOKEN PREDICTION FOR EDGE ARCHITECTURES

Anonymous authors
Paper under double-blind review

ABSTRACT

Speculative decoding (SD) accelerates autoregressive generation by drafting multiple tokens and verifying them in a single model forward pass, exploiting the memory-bound nature of LLM inference (Leviathan et al., 2023; Chen et al., 2023). Modern SD techniques such as Medusa and Eagle are designed to maximize operational intensity, so they are highly effective on GPUs but not edge devices (Cai et al., 2024; Li et al., 2024). Additionally, their draft heads can significantly increase parameter count, undermining the benefits of small architectures. In this work, we benchmark Medusa's independent-head and Eagle's sequential-feature architectures on Gemma 3 270M (Google DeepMind, 2024), comparing performance between Nvidia A100 GPUs and Apple M4 Pro using the MLX inference engine (Apple, 2024; Hannun et al., 2023). We find that deep speculation trees optimized for GPUs (Miao et al., 2024) shift the bottleneck on Apple Silicon: expanded attention masks and verification compute outweigh reduced memory traffic, causing 3–5× slowdowns (Williams et al., 2009). Motivated by this failure mode, we evaluate a minimalist recipe—shallow, single-step speculation with a compact dense draft module that reuses the base LM head. This variant, *Lite MTP*, yields a 7–28% throughput improvement over greedy decoding on MLX with only 0.3% parameter overhead.

1 INTRODUCTION

The proliferation of on-device AI has driven the adoption of Small Language Models (SLMs) such as Gemma 3 270M as alternatives or supplements to cloud-based inference for easy workloads. However, widespread adoption is still limited by high latency that breaks the interactiveness of their applications. Furthermore, increasing response speeds expands the frontier of what model sizes are viable to be run locally. While SLMs have significantly reduced parameter counts, the autoregressive nature of transformer generation is fundamentally memory-bound (Williams et al., 2009). To balance the memory bottleneck and increase speed, Speculative Decoding (SD) with Multi-Token Prediction (MTP) has emerged as a popular paradigm (Leviathan et al., 2023; Chen et al., 2023; Gloeckle et al., 2024). Methods like Medusa's independent heads (Cai et al., 2024) and Eagle's sequential feature extraction (Li et al., 2024) increase the operational intensity of the decoding process to better utilize GPUs (e.g., Nvidia A100s). They draft multiple future tokens in a single forward pass to convert a portion of the memory-bottleneck to a compute bottleneck, while being lossless. On GPUs, the cost of these deep multi-token speculation trees (Miao et al., 2024) is effectively hidden due to their massively parallel compute optimized nature. This is in stark contrast to mobile chips because the A100 40GB has an FP16 Machine Balance (ratio of peak FLOPs to peak memory bandwidth) of approximately 201 FLOPs/byte (NVIDIA, 2020), whereas the Apple M4 Pro chip's GPU only has about 62 FLOPs/byte (Apple, 2024), more than 3× less. In this work, we run 3 core experiments to understand and fix the mismatch between current MTP practices and on-device inference. First, we implement and train Medusa and Eagle on Gemma 3 270M using self-generated trajectories, reproducing strong speedups for the standard GPU setting. Second, we transfer and evaluate the models' optimized decode performance on Apple Silicon (Hannun et al., 2023). Lastly, we identify that the on-device bottleneck stems from deep speculation trees, which inflate attention and verification compute to the point where the overhead dominates the increase in throughput. Thus, we devise a simple and hardware-aware recipe centered on shallow, single-step speculation that keeps compute small enough to accelerate inference.

Contributions.

Table 1: Parameter overhead on Gemma 3 270M: dense Medusa heads bloat size (256k vocab), while our draft heads add negligible overhead.

Model Configuration	Head Type	Add. Params	Total Size	Increase
Base Model	—	—	270M	0%
Medusa (Standard, 1 Head)	Dense (d=640)	∼171M	441M	+63%
Medusa (Standard, 3 Heads)	Dense (d=640)	∼786M	1.05B	+189%
Eagle (Ours, Depth 3)	Fusion + Layer	∼6.5M	276.5M	+2.4%
Lite MTP (Ours)	**2x Dense (d=640)**	**∼820k**	**270.8M**	**+0.3%**

- We reproduce GPU speedups for Medusa and Eagle on Gemma 3 270M and quantify how throughput scales with deeper speculation trees.
- We show that these GPU-favored configurations can be 3–5× slower on Apple Silicon under MLX, and we profile the bottleneck shift to backbone/verification compute from expanded tree attention.
- We propose Lite MTP, a minimal Medusa variant that reuses the base LM head and favors shallow trees, achieving 7–28% throughput gains on MLX with 0.3% parameter overhead.

2 Methodology

We use Gemma 3 270M as our base SLM because it is a common target for on-device deployment and still has enough breadth to warrant and be capable of speculative decoding. To investigate the hardware-specific performance spectrum, we implement two foundational MTP paradigms (Medusa and Eagle) and our minimalist Medusa variant (Lite MTP) (Cai et al., 2024; Li et al., 2024).

2.1 Architecture Variants

Medusa Medusa (independent heads) first introduced the MTP speculative decoding paradigm to LLMs. Medusa augments the base model with K independent, lightweight draft heads. Concretely, each head consists of a single linear layer followed by a SiLU activation and an LM-head, predicting future tokens $t+1, t+2, \ldots, t+k$ conditioned on the backbone's final hidden state h_t. Once predicted, Medusa relies on a large tree-attention mask (up to 80 nodes) during the verification pass to maximize token acceptance on GPU (Cai et al., 2024; Miao et al., 2024).

Inference Verification At step t, the backbone produces h_t (Leviathan et al., 2023). Heads use this to draft the probability distribution of candidates that we arrange into a greedy tree (Cai et al., 2024). We verify the tree in one backbone pass using tree-attention masks (Miao et al., 2024), and accept only tokens matching the backbone's greedy choice (Chen et al., 2023). We then repeat the MTP process from the last accepted state (Cai et al., 2024).

Eagle Eagle replaces independent heads with an autoregressive 1-layer transformer over features from early, middle, and later backbone layers, producing depth-k draft trees (Li et al., 2024). This can improve draft coherence, but increases drafting compute, especially for smaller models. Interestingly, Eagle does not have an LM head of its own and uses the base model's. This was the inspiration for our method to also reuse it.

Ours: Lite MTP For SLMs like Gemma 3 270M, standard Medusa heads are memory-prohibitive because the vocabulary is large (256k). A single vocab-sized LM head adds ∼170M parameters. The standard 5 Medusa heads nearly quadruples model size (Table 1), undermining the point of a 270M backbone.

We therefore propose *Lite MTP*, a minimalist Medusa variant that drafts candidates with a compact dense module (two 640×640 layers) and reuses the base model's LM head for token scoring. We evaluated both a LoRA adapter on the LM head (rank 64) (Hu et al., 2021) and a no-LoRA design. We found the differences to be small (Table **??**), so we adopt the simpler no-LoRA configuration for edge deployment, adding only 0.3% parameters (Table 1).

2.2 Training Rigor and Loss

To ensure draft heads align with the base model's distribution, we moved beyond generic corpora like FineWeb-EDU (Penedo et al., 2024) and used self-generated trajectories. Specifically, we generated 100k completions by prompting Gemma 3 270M with the OpenHermes 2.5 dataset (Teknium,

Table 2: Recall@k for Medusa head configurations on the self-distilled dataset. KL loss dramatically outperforms CE, while head type differences are minimal.

		Head 0		**Head 1**		**Head 2**	
Loss	**LM Head**	top1	top10	top1	top10	top1	top10
CE	Full	.055	.384	.060	.373	.067	.359
CE	NoLoRA	.053	.341	.052	.312	.056	.300
CE	LoRA-64	.051	.371	.062	.361	.066	.350
KL	NoLoRA	.373	.686	.224	.501	.151	.412
KL	LoRA-64	**.419**	**.752**	**.258**	**.580**	**.182**	**.487**

Table 3: GPU Decoding Throughput on GSM8K (left) and HumanEval (right). Tok/Step is the average total tokens accepted per decoding step (including the 1 token from the backbone). The baseline tok/s are 35.2 and 33.6 for GSM8K and HumanEval respectively. KL divergence training improves Medusa acceptance rates, while Eagle benefits from increased depth and accuracy.

GSM8K

Method	**Config**	**Tok/Step**	**Speedup**
Baseline	—	1.00	1.00×
Medusa	2 layers	1.54	1.09×
Medusa	KL + 2 layers	2.15	1.52×
EAGLE	Depth 1	1.94	1.38×
EAGLE	Depth 2	2.78	1.38×
EAGLE	Depth 3	3.42	1.52×

HumanEval

Method	**Config**	**Tok/Step**	**Speedup**
Baseline	—	1.00	1.00×
Medusa	2 layers	1.35	1.01×
Medusa	KL + 2 layers	1.96	1.47×
EAGLE	Depth 1	1.94	1.44×
EAGLE	Depth 2	2.89	1.50×
EAGLE	Depth 3	3.68	1.64×

2023). This keeps training in-distribution and lets us use KL-based distillation against the backbone. While we initially used Cross-Entropy (CE) loss for simplicity (as in Medusa-1), we found Kullback-Leibler (KL) Divergence consistently improved alignment, matching the backbone's soft-target distribution (rather than hard labels) increases acceptance rates and downstream throughput.

2.3 Evaluation Setup

We measure decoding throughput (tokens per second at batch size 1) on an Nvidia A100 and Apple M4 Pro via MLX (Hannun et al., 2023), sweeping draft tree size (speculative candidates) to capture hardware regime changes. We assume that MLX's implementation is near-optimal and approximates true hardware performance on Apple Silicon, as it is the standard framework for on-device inference benchmarking. Large trees are empirically effective on GPU, while on-device inference prefers shallow speculation (Figure 2).

3 Experiments & Results

3.1 GPU Baseline

We first validate that multi-token prediction (MTP) provides meaningful throughput gains for small models in the standard GPU setting. We trained Medusa and Eagle variants on our self-distilled trajectories and evaluated them under decoding throughput (Cai et al., 2024; Li et al., 2024). Across both GSM8K and HumanEval, stronger draft alignment from KL divergence loss correlates with higher acceptance and higher throughput, as switching Medusa to KL-only training increases throughput from 35.9 to 49.5 tps on GSM8K and 33.6 to 49.2 tps on HumanEval (Tables **??**–**??**) (Cobbe et al., 2021; Chen et al., 2021). Eagle shows a clear preference for depth on HumanEval (48.5, 50.5, 56.2 tps at depth 1/2/3), but at 270M scale the heavier drafting module limits how much of its higher token yield translates into end-to-end speed.

3.2 Edge Failure

We then port the same MTP setup to Apple Silicon using MLX, again measuring decoding throughput (Hannun et al., 2023). However, the configurations that excelled on GPU were dramatically slower on-edge, the tok/s were between 3–5× slower relative to the MLX baseline. Examining the profiling in depth reveals that as tree size grows, verification becomes dominated by backbone and LM-head compute, and the expanded tree attention makes the backbone pass disproportionately expensive. Concretely, increasing tree size to 80 raises backbone cost by 3.17× on M4 Pro, while it increases only 1.12× on A100 (Apple, 2024; NVIDIA, 2020). In this regime, most time is spent in the backbone pass, so the extra verification compute overwhelms the throughput gains from drafting more candidates.

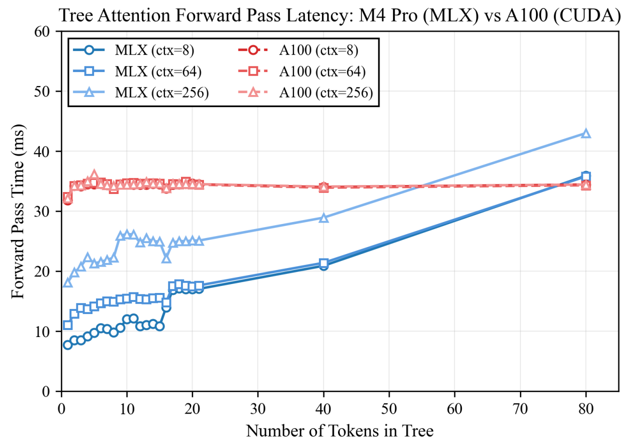

Figure 1: Profiling on MLX (M4 Pro) vs A100: tree size shifts backbone/verification cost.

Table 4: MLX results on GSM8K and HumanEval. Tok/Step = average tokens accepted per decoding step. Lite MTP is fastest.

GSM8K				HumanEval			
Mode	**Tok/s**	**Tok/Step**	**Speedup**	**Mode**	**Tok/s**	**Tok/Step**	**Speedup**
Baseline	218.4	1.00	1.00×	Baseline	162.1	1.00	1.00×
Lite MTP	**234.4**	1.39	**1.07**×	Lite MTP	**208.6**	1.36	**1.29**×
Lite MTP-3tok	217.8	1.41	1.00×	Lite MTP-3tok	201.1	1.35	1.24×
Lite MTP-Depth2	215.9	1.48	0.99×	Lite MTP-Depth2	201.1	1.43	1.24×

Table 5: Profiling Comparison (Context = 16). Backbone cost scales 3.17× on MLX vs 1.12× on GPU as tree size grows from 1 to 80.

	MLX (M4 Pro)			**GPU (A100)**		
Tree Size	**Backbone**	**Verify**	**Tok/s**	**Backbone**	**Verify**	**Tok/s**
1	3.32ms	4.62ms	316.1	31.66ms	31.57ms	18.5
2	3.56ms	5.11ms	299.5	38.49ms	35.67ms	19.0
4	3.66ms	5.70ms	279.2	35.30ms	35.60ms	18.8
8	3.69ms	5.74ms	331.5	35.56ms	35.29ms	21.2
16	7.66ms	10.60ms	250.2	35.56ms	35.37ms	21.4
80	10.53ms	16.29ms	223.7	35.71ms	36.13ms	22.2

3.3 Minimalist Speculation

Given the edge failure mode, we use sweeps over tree size (and context) to find the compute-optimal regime on-device. In an idealized sweep, the best tree size for M4 Pro is around 4, but this does not fully capture practical overheads in MLX (e.g., cache behavior when re-arranging KV state and Python overhead for restructuring tree attention). Empirically, tree size 2 is the most robust, our results show that it improves throughput from 218.4 to 234.4 tps (+7.3%) on GSM8K, while larger trees tend to degrade performance (Cobbe et al., 2021). On HumanEval, tree size 2 improves 162.1 to 208.6 tps (+28.6%), with larger trees still positive but weaker (+24%) (Chen et al., 2021). Given the consistent advantage of minimal trees, we present Lite MTP: a hardware-aware recipe for on-device inference using single-token, single-head speculation, where we keep verification overhead small enough to realize net speedups on Apple M4 Pro (Apple, 2024).

4 Conclusion

We demonstrate that Multi-Token Prediction strategies optimized for GPUs, achieving up to 1.52× speedup on GSM8K and 1.64× on HumanEval, fail catastrophically on edge devices, resulting in 3-5× slowdowns due to insufficient FLOPs-to-bandwidth ratios to hide deep verification tree overhead. Lite MTP, our hardware-aware solution using single-token single-head speculation with compact dense heads, restores speedups of 1.07× on GSM8K and 1.29× on HumanEval on Apple M4 Pro while adding only 0.3% parameter overhead (820k parameters), compared to 63–189% for standard Medusa heads. This minimal footprint preserves the memory efficiency that makes SLMs viable for on-device deployment. Our findings suggest that effective edge speculation requires minimizing verification overhead rather than maximizing theoretical token yield.

References

Apple. M4 pro chip specifications. Apple Newsroom/Technical Specs, 2024.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, 2024.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.

Google DeepMind. Gemma 3 technical report. Technical report, 2024.

Awni Hannun et al. Mlx: An array framework for apple silicon. GitHub Repository, 2023.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023.

Yuhui Li, Fangyun Fan, Wei Chu, and Yingce Li. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification. *arXiv preprint arXiv:2305.09781*, 2024.

NVIDIA. Nvidia a100 tensor core gpu architecture. NVIDIA Whitepaper, 2020.

Guilherme Penedo, Hynek Kydlicek, Alessandro Cappelli, David Saez de Ocariz Borde, Marco Olocco, and Daniel Villanova del Moral. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

Teknium. Openhermes 2.5: An open dataset of synthetic data for generalist llms. HuggingFace, 2023.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.